
Chapter 6

Formal Relational Query Languages



Formal Relational Query Languages

- ▶ In Chapters 2 through 5 we introduced the relational model and covered SQL in great detail.
- ▶ In this chapter we present the formal model upon which SQL as well as other relational query languages are based. We cover three formal languages:
 - ▶ **The relational algebra**
 - ▶ It forms the basis of the widely used SQL query language.
 - ▶ **The tuple relational calculus**
 - ▶ **The domain relational calculus**
 - ▶ The above two are declarative query languages based on *mathematical logic*.



Outline

- ▶ **Relational Algebra**
- ▶ **Tuple Relational Calculus**
- ▶ **Domain Relational Calculus**



The Relational Algebra

- ▶ The **relational algebra** (关系代数) is a **procedural query language**.
- ▶ It consists of *a set of operations* that take *one or two relations* as input and produce *a new relation* as their result.
- ▶ Six basic operators:
 - ▶ **Select:** σ
 - ▶ **Project:** Π
 - ▶ **Union:** $\cup$
 - ▶ **Set Difference:** $-$
 - ▶ **Cartesian Product:** $\times$
 - ▶ **Rename:** ρ



The Select Operation

- ▶ The **select** operation *selects tuples that satisfy a given predicate*.
- ▶ Notation: $\sigma_p(r)$, p is called the **selection predicate**
- ▶ Defined as:

$$\sigma_p(r) = \{t \mid t \in r \text{ and } p(t)\}$$

where p is a formula in propositional calculus consisting of **terms** connected by : $\wedge$ (**and**), $\vee$ (**or**), $\neg$ (**not**)

- ▶ Each **term** is one of:

$$\langle \text{attribute} \rangle \text{ op } \langle \text{attribute} \rangle / \langle \text{constant} \rangle$$

where op is one of $=, \neq, >, \geq, <, \leq$



The Select Operation - Example

- ▶ Relation r

A	B	C	D
α	α	1	7
α	β	5	7
β	β	12	3
β	β	23	10

- ▶ $\sigma_{A=B \wedge D > 5}(r)$

A	B	C	D
α	α	1	7
β	β	23	10



The Select Operation - Example

- ▶ $\sigma_{dept_name = \text{"Physics"}}(instructor)$

ID	name	dept_name	salary
22222	Einstein	Physics	95000
33456	Gold	Physics	87000

Figure 6.2 Result of $\sigma_{dept_name = \text{"Physics"}}(instructor)$.

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure 6.1 The *instructor* relation.

- ▶ The selection predicate may include *comparisons between two attributes*:
 - ▶ To find all *departments* whose *name* is the same as their *building name*:

$$\sigma_{dept_name = building}(department)$$



The Project Operation

- ▶ The project operation is a **unary (一元) operation** that *returns its argument relation with certain attributes left out.*
- ▶ Notation: $\Pi_{A_1, A_2, \dots, A_k}(r)$ where A_1, A_2 are *attribute names* and r is a *relation name*.
- ▶ The result is defined as *the relation of k columns obtained by erasing the columns that are not listed.*
- ▶ Duplicate rows removed from result, since relations are sets.



The Project Operation - Example

► Relation r :

A	B	C
α	10	1
α	20	1
β	30	1
β	40	2

► $\Pi_{A,C}(r)$

A	C
α	1
α	1
β	1
β	2

=

A	C
α	1
β	1
β	2



The Project Operation - Example

To list all *instructors*' *ID*, *name*, and *salary* without *dept_name*:

$$\Pi_{ID, name, salary}(instructor)$$

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Figure 6.1 The *instructor* relation.

<i>ID</i>	<i>name</i>	<i>salary</i>
10101	Srinivasan	65000
12121	Wu	90000
15151	Mozart	40000
22222	Einstein	95000
32343	El Said	60000
33456	Gold	87000
45565	Katz	75000
58583	Califieri	62000
76543	Singh	80000
76766	Crick	72000
83821	Brandt	92000
98345	Kim	80000

Figure 6.3 Result of $\Pi_{ID, name, salary}(instructor)$.



Composition of Relational Operations

- ▶ *Relational-algebra operations* can be composed together into a **relational-algebra expression**.
- ▶ Composing relational-algebra operations into relational-algebra expressions is just like composing *arithmetic operations* (such as $+$, $-$, $*$, and $\div$) into arithmetic expressions.
- ▶ Example:

Find the *name* of all *instructors* in the Physics Department

$$\Pi_{name} (\sigma_{dept_name = "Physics"} (instructor))$$



The Union Operation

▶ Notation: $r \cup s$

▶ Defined as:

$$r \cup s = \{t \mid t \in r \text{ or } t \in s\}$$

▶ For $r \cup s$ to be **valid**.

1. r, s must have the **same arity** (*same number of attributes*).
2. The domains of the i^{th} attribute of r and the i^{th} attribute of s must be **the same**, for all i .

▶ *Note* : r and s can be either **database relations** or **temporary relations** that are the result of relational-algebra expressions.



The Union Operation - Example

► Relations r, s :

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

► $r \cup s$:

A	B
α	1
α	2
β	1
β	3



The Union Operation - Example

To find all *courses taught* in the Fall 2009 semester, the Spring 2010 semester, or both:

$$\Pi_{course_id}(\sigma_{semester="Fall" \wedge year=2009}(section)) \cup \Pi_{course_id}(\sigma_{semester="Spring" \wedge year=2010}(section))$$

course_id	sec_id	semester	year	building	room_number	time_slot_id
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

course_id
CS-101
CS-315
CS-319
CS-347
FIN-201
HIS-351
MU-199
PHY-101

Figure 6.4 The *section* relation.

- ▶ **Duplicate values** such as CS-101, which is offered in both semesters, are replaced by a single occurrence.



The Set-Difference Operation

- ▶ The **set-difference** operation, denoted by $-$, allows us to find tuples that are in one relation but are not in another.
- ▶ Notation: $r - s$
- ▶ Defined as:

$$r - s = \{t \mid t \in r \text{ and } t \notin s\}$$

- ▶ Set differences *must be taken between compatible relations*.
 - ▶ r and s must have the **same arity** (*same number of attributes*).
 - ▶ The domains of the i^{th} attribute of r and the i^{th} attribute of s must be the same, for all i .



Set-Difference Operation - Example

- Relations r, s :

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

- $r - s$:

A	B
α	1
β	1

- To find all *courses* taught in the Fall 2009 semester, but not in the Spring 2010 semester:

$\Pi_{course_id}(\sigma_{semester="Fall" \wedge year=2009}(section)) -$

$\Pi_{course_id}(\sigma_{semester="Spring" \wedge year=2010}(section))$



The Cartesian-Product Operation

- ▶ The **Cartesian-product** operation, denoted by a cross ($\times$), allows us to *combine information from any two relations*.
- ▶ Notation: $r \times s$
- ▶ Defined as:

$$r \times s = \{tq \mid t \in r \text{ and } q \in s\}$$

▶ Example

- ▶ The relation schema for $r = \text{instructor} \times \text{teaches}$ is:
(instructor.ID, instructor.name, instructor.dept_name, instructor.salary, teaches.ID, teaches.course_id, teaches.sec_id, teaches.semester, teaches.year)
- ▶ For those attributes that appear in only one of the two schemas, we shall usually drop the relation-name prefix:
(instructor.ID, name, dept_name, salary, teaches.ID, course_id, sec_id, semester, year)



The Cartesian-Product Operation

► Relations r, s :

A	B
α	1
β	2

r

C	D	E
α	10	a
β	10	a
β	20	b
γ	10	b

s

In general, if we have relations $r_1(R_1)$ and $r_2(R_2)$, then $r_1 \times r_2$ is a relation whose schema is the **concatenation** of R_1 and R_2 .

► $r \times s$:

A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b

Relation R contains all tuples t for which there is a tuple t_1 in r_1 and a tuple t_2 in r_2 for which $t[R_1] = t_1[R_1]$ and $t[R_2] = t_2[R_2]$.



Cartesian-Product Operation - Example

► Result of $instructor \times teaches$:

<i>inst.ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>	<i>teaches.ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>
10101	Srinivasan	Physics	95000	10101	CS-101	1	Fall	2009
10101	Srinivasan	Physics	95000	10101	CS-315	1	Spring	2010
10101	Srinivasan	Physics	95000	10101	CS-347	1	Fall	2009
10101	Srinivasan	Physics	95000	10101	FIN-201	1	Spring	2010
10101	Srinivasan	Physics	95000	15151	MU-199	1	Spring	2010
10101	Srinivasan	Physics	95000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
12121	Wu	Physics	95000	10101	CS-101	1	Fall	2009
12121	Wu	Physics	95000	10101	CS-315	1	Spring	2010
12121	Wu	Physics	95000	10101	CS-347	1	Fall	2009
12121	Wu	Physics	95000	10101	FIN-201	1	Spring	2010
12121	Wu	Physics	95000	15151	MU-199	1	Spring	2010
12121	Wu	Physics	95000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
15151	Mozart	Physics	95000	10101	CS-101	1	Fall	2009
15151	Mozart	Physics	95000	10101	CS-315	1	Spring	2010
15151	Mozart	Physics	95000	10101	CS-347	1	Fall	2009
15151	Mozart	Physics	95000	10101	FIN-201	1	Spring	2010
15151	Mozart	Physics	95000	15151	MU-199	1	Spring	2010
15151	Mozart	Physics	95000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
22222	Einstein	Physics	95000	10101	CS-101	1	Fall	2009
22222	Einstein	Physics	95000	10101	CS-315	1	Spring	2010
22222	Einstein	Physics	95000	10101	CS-347	1	Fall	2009
22222	Einstein	Physics	95000	10101	FIN-201	1	Spring	2010
22222	Einstein	Physics	95000	15151	MU-199	1	Spring	2010
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...



Cartesian-Product Operation - Example

- ▶ To find the *names* of all *instructors* in the Physics *department* together with the *course id* of all *courses* they taught:

$\Pi_{name, course_id} (\sigma_{instructor.ID = teaches.ID} (\sigma_{dept_name = \text{“Physics”}} (instructor \times teaches)))$

- ▶ Note: there is often **more than one way to write a query** in relational algebra.

$\Pi_{name, course_id} (\sigma_{instructor.ID = teaches.ID} ((\sigma_{dept_name = \text{“Physics”}} (instructor)) \times teaches))$

<i>name</i>	<i>course_id</i>
Einstein	PHY-101

Figure 6.10 Result of

$\Pi_{name, course_id} (\sigma_{instructor.ID = teaches.ID} (\sigma_{dept_name = \text{“Physics”}} (instructor \times teaches)))$



The Rename Operation

- ▶ The **rename** operator allows us to *name*, and therefore to *refer to, the results of relational-algebra expressions*.
- ▶ Allows us to *refer to a relation by more than one name*.
- ▶ Given a relational-algebra expression E ,

$$\rho_x(E)$$

returns *the expression E under the name X* .

- ▶ A second form: assume a relational-algebra expression E has arity n ,

$$\rho_{x(A_1, A_2, \dots, A_n)}(E)$$

returns *the result of expression E under the name X , and with the attributes renamed to $A_1, A_2, \dots, A_n$* .



The Rename Operation - Example

- ▶ Find the highest *salary* in the university.
 - ▶ **Step 1:** Compute first a temporary relation consisting of those salaries that are *not* the largest.

$$\Pi_{instructor.salary} (\sigma_{instructor.salary < d.salary}(instructor \times \rho_d(instructor)))$$

- ▶ **Step 2:** Take the set difference between the relation $\Pi_{salary}(instructor)$ and the temporary relation just computed to obtain the result.

$$\Pi_{salary}(instructor) -$$

$$\Pi_{instructor.salary} (\sigma_{instructor.salary < d.salary}(instructor \times \rho_d(instructor)))$$



Formal Definition of the Relational Algebra

- ▶ A **basic expression** in the relational algebra consists of either one of the following:
 - ▶ *A relation in the database*
 - ▶ *A constant relation*
 - ▶ A constant relation is written by listing its tuples within $\{ \}$, for example $\{ (22222, \text{Einstein}, \text{Physics}, 95000), (76543, \text{Singh}, \text{Finance}, 80000) \}$.
- ▶ Let E_1 and E_2 be relational-algebra expressions. Then, the following are **all relational-algebra expressions**:
 - ▶ $E_1 \cup E_2$
 - ▶ $E_1 - E_2$
 - ▶ $E_1 \times E_2$
 - ▶ $\sigma_P(E_1)$, where P is a predicate on attributes in E_1
 - ▶ $\Pi_S(E_1)$, where S is a list consisting of some of the attributes in E_1
 - ▶ $\rho_x(E_1)$, where x is the new name for the result of E_1



Additional Relational-Algebra Operations

- ▶ The fundamental operations of the relational algebra are sufficient to express any relational-algebra query. However, if we restrict ourselves to just the fundamental operations, certain *common queries are lengthy to express*.
- ▶ Therefore, we define **additional operations (附加运算)** that *do not add any power to the algebra, but simplify common queries*.
 - ▶ Set intersection
 - ▶ Natural join
 - ▶ Assignment
 - ▶ Outer join



The Set-Intersection Operation

▶ Notation: $r \cap s$

▶ Defined as:

$$r \cap s = \{t \mid t \in r \text{ and } t \in s\}$$

▶ Note:

▶ $r \cap s = r - (r - s)$

▶ r and s are compatible relations.

▶ Example:

▶ Find the set of all *courses* taught in both the Fall 2009 and the Spring 2010 semesters.

$\Pi_{course_id}(\sigma_{semester="Fall" \wedge year=2009}(section)) \cap$

$\Pi_{course_id}(\sigma_{semester="Spring" \wedge year=2010}(section))$



The Natural-Join Operation

- ▶ Notation: $r \bowtie s$
- ▶ Consider two relations $r(R)$ and $s(S)$. $r \bowtie s$ is a relation on schema $R \cup S$ formally defined as:

$$r \bowtie s = \Pi_{R \cup S} (\sigma_{r.A1=s.A1 \wedge r.A2=s.A2 \wedge \dots \wedge r.An=s.An} (r \times s))$$

where $R \cap S = \{A_1, A_2, \dots, A_n\}$.

- ▶ Note:
 - ▶ If $R \cap S = \emptyset$, then $r \bowtie s = r \times s$.
 - ▶ The *natural-join operation* forms a Cartesian product of its two arguments, performs a selection forcing equality on those attributes that appear in both relation schemas, and finally removes duplicate attributes.



The Natural-Join Operation - Example

► Relations r, s :

A	B	C	D
α	1	α	a
β	2	γ	a
γ	4	β	b
α	1	γ	a
δ	2	β	b

r

B	D	E
1	a	α
3	a	β
1	a	γ
2	b	δ
3	b	ϵ

s

► $r \bowtie s$:

A	B	C	D	E
α	1	α	a	α
α	1	α	a	γ
α	1	γ	a	α
α	1	γ	a	γ
δ	2	β	b	δ



The Natural-Join Operation - Example

- ▶ Computing *instructor natural join teaches* considers only those pairs of tuples where *both the tuple from instructor and the tuple from teaches have the same value on the common attribute ID*.
- ▶ The **order** in which the attributes are listed:
 - ▶ **First** the attributes *common to the schemas of both relations*;
 - ▶ **Second** those attributes *unique to the schema of the first relation*;
 - ▶ **Finally** those attributes *unique to the schema of the second relation*.

ID	name	dept_name	salary	course_id	sec_id	semester	year
10101	Srinivasan	Comp. Sci.	65000	CS-101	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000	CS-315	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	CS-347	1	Fall	2009
12121	Wu	Finance	90000	FIN-201	1	Spring	2010
15151	Mozart	Music	40000	MU-199	1	Spring	2010
22222	Einstein	Physics	95000	PHY-101	1	Fall	2009
32343	El Said	History	60000	HIS-351	1	Spring	2010
45565	Katz	Comp. Sci.	75000	CS-101	1	Spring	2010
45565	Katz	Comp. Sci.	75000	CS-319	1	Spring	2010
76766	Crick	Biology	72000	BIO-101	1	Summer	2009
76766	Crick	Biology	72000	BIO-301	1	Summer	2010
83821	Brandt	Comp. Sci.	92000	CS-190	1	Spring	2009
83821	Brandt	Comp. Sci.	92000	CS-190	2	Spring	2009
83821	Brandt	Comp. Sci.	92000	CS-319	2	Spring	2010
98345	Kim	Elec. Eng.	80000	EE-181	1	Spring	2009

Figure 6.14 The natural join of the *instructor* relation with the *teaches* relation.

The Natural-Join Operation - Example

- ▶ To find the *names* of all *instructors* in the Comp. Sci. *department* together with the *course titles* of all the *courses* that the *instructors teach*:

$\Pi_{name, title} (\sigma_{dept_name = \text{"Comp. Sci."}} (instructor \bowtie teaches \bowtie course))$

- ▶ The natural join is **associative**. The following two is equivalent:
 - ▶ $(instructor \bowtie teaches) \bowtie course$
 - ▶ $instructor \bowtie (teaches \bowtie course)$
- ▶ The natural join is **commutative**. The following two is equivalent:
 - ▶ $instructor \bowtie teaches$
 - ▶ $teaches \bowtie instructor$



A Variant : Theta Join

► Notation: $r \bowtie_{\theta} s$

► Defined as :

$$r \bowtie_{\theta} s = \sigma_{\theta} (r \times s)$$

where θ is a *predicate* on attributes in the schema $R \cup S$.

► The *theta join* operation allows us to *combine a selection and a Cartesian product into a single operation*.



The Assignment Operation

- ▶ The **assignment operation** ($\leftarrow$) provides a convenient way to express complex queries.
- ▶ A query can be written as *a sequential program* consisting of a series of assignments followed by an expression whose value is displayed as the result of the query.
- ▶ For relational-algebra queries, assignment must always be made to a *temporary relation variable*.



The Assignment Operation

- ▶ To illustrate assignment operation, consider the definition of the natural-join operation. We could write $r \bowtie s$ as:

$$\textit{temp1} \leftarrow r \times s$$

$$\textit{temp2} \leftarrow \sigma_{r.A1=s.A1 \wedge r.A2=s.A2 \wedge \dots \wedge r.An=s.An}(\textit{temp1})$$

$$\textit{result} \leftarrow \Pi_{R \cup S}(\textit{temp2})$$

- ▶ The assignment operation does not provide any additional power to the algebra, but it is **convenient** at times to write a relational-algebra expression by assigning parts of it to temporary relation variables.



Outer Join Operations

- ▶ An extension of the join operation that *avoids loss of information*.
- ▶ The **outer join** operation works in a manner similar to the natural join operation we have already studied, but *preserves those tuples that would be lost in an join by creating tuples in the result containing null values*.
- ▶ The outer-join operations may generate results containing *null* values. And the relational-algebra operations deal with null values in the same way as SQL.



Outer Join Operations

- ▶ **Three forms of outer join**

- ▶ **Left outer join** ($\bowtie\lt$)

- ▶ It takes all tuples in the left relation that did not match with any tuple in the right relation, pads the tuples with *null* values for all other attributes from the right relation, and adds them to the result of the natural join.

- ▶ **Right outer join** ($\lt\bowtie$)

- ▶ It is symmetric with the left outer join.

- ▶ **Full outer join** ($\bowtie\lt\bowtie$)

- ▶ It does both the left and right outer join operations.



Outer Join Operations – Example

► Result of *instructor* ⋈ *teaches*

ID	name	dept_name	salary	course_id	sec_id	semester	year
10101	Srinivasan	Comp. Sci.	65000	CS-101	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000	CS-315	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	CS-347	1	Fall	2009
12121	Wu	Finance	90000	FIN-201	1	Spring	2010
15151	Mozart	Music	40000	MU-199	1	Spring	2010
22222	Einstein	Physics	95000	PHY-101	1	Fall	2009
32343	El Said	History	60000	HIS-351	1	Spring	2010
33456	Gold	Physics	87000	null	null	null	null
45565	Katz	Comp. Sci.	75000	CS-101	1	Spring	2010
45565	Katz	Comp. Sci.	75000	CS-319	1	Spring	2010
58583	Califieri	History	62000	null	null	null	null
76543	Singh	Finance	80000	null	null	null	null
76766	Crick	Biology	72000	BIO-101	1	Summer	2009
76766	Crick	Biology	72000	BIO-301	1	Summer	2010
83821	Brandt	Comp. Sci.	92000	CS-190	1	Spring	2009
83821	Brandt	Comp. Sci.	92000	CS-190	2	Spring	2009
83821	Brandt	Comp. Sci.	92000	CS-319	2	Spring	2010
98345	Kim	Elec. Eng.	80000	EE-181	1	Spring	2009



Outer Join Operations – Example

► Result of *instructor* $\bowtie$ *teaches*

ID	course_id	sec_id	semester	year	name	dept_name	salary
10101	CS-101	1	Fall	2009	Srinivasan	Comp. Sci.	65000
10101	CS-315	1	Spring	2010	Srinivasan	Comp. Sci.	65000
10101	CS-347	1	Fall	2009	Srinivasan	Comp. Sci.	65000
12121	FIN-201	1	Spring	2010	Wu	Finance	90000
15151	MU-199	1	Spring	2010	Mozart	Music	40000
22222	PHY-101	1	Fall	2009	Einstein	Physics	95000
32343	HIS-351	1	Spring	2010	El Said	History	60000
33456	null	null	null	null	Gold	Physics	87000
45565	CS-101	1	Spring	2010	Katz	Comp. Sci.	75000
45565	CS-319	1	Spring	2010	Katz	Comp. Sci.	75000
58583	null	null	null	null	Califieri	History	62000
76543	null	null	null	null	Singh	Finance	80000
76766	BIO-101	1	Summer	2009	Crick	Biology	72000
76766	BIO-301	1	Summer	2010	Crick	Biology	72000
83821	CS-190	1	Spring	2009	Brandt	Comp. Sci.	92000
83821	CS-190	2	Spring	2009	Brandt	Comp. Sci.	92000
83821	CS-319	2	Spring	2010	Brandt	Comp. Sci.	92000
98345	EE-181	1	Spring	2009	Kim	Elec. Eng.	80000



Outer Join Operations

- ▶ **Outer join** can be expressed *using basic relational-algebra operations*.

- ▶ For example: $r \bowtie s$ can be written as

$$(r \bowtie s) \cup (r - \Pi_{R \cup S}(r \bowtie s)) \times \{(null, \dots, null)\}$$

where the constant relation $\{(null, \dots, null)\}$ is on the schema **$S - R$** .



Extended Relational-Algebra Operations

- ▶ We now describe relational-algebra operations that provide the ability to *write queries that cannot be expressed using the basic relational-algebra operations.*
- ▶ **Extended relational-algebra operations:**
 - ▶ **Generalized Projection**
 - ▶ **Aggregate Functions**



Generalized Projection

- ▶ **Generalized-projection** operation extends the **projection operation** by allowing operations such as *arithmetic and string functions* to be used in the projection list.

$$\Pi_{F_1, F_2, \dots, F_n}(E)$$

- ▶ E is any relational-algebra expression;
- ▶ Each of $F_1, F_2, \dots, F_n$ are an *arithmetic expressions* involving *constants* and *attributes* in the schema of E .
- ▶ Example: Given the relation *instructor*($ID, name, dept_name, salary$), give the $ID, name, dept_name$, and the monthly salary of each instructor:

$$\Pi_{ID, name, dept_name, salary \div 12} (instructor)$$



Aggregate Functions and Operations

- ▶ **Aggregate function** takes a collection of values and returns a single value as a result.

avg: average value

min: minimum value

max: maximum value

sum: sum of values

count: number of values

- ▶ **Aggregation operation** in relational algebra

$$G_1, G_2, \dots, G_n \quad \mathcal{G} \quad F_1(A_1), F_2(A_2), \dots, F_n(A_n) (E)$$

- ▶ E is any relational-algebra expression;
- ▶ $G_1, G_2, \dots, G_n$ is a list of attributes on which to group (can be empty);
- ▶ Each F_i is an aggregate function; Each A_i is an attribute name.
- ▶ Note: Some books/articles use γ instead of $\mathcal{G}$



Aggregation Operation - Example

▶ Relation r :

A	B	C
α	α	7
α	β	7
β	β	3
β	β	10

▶ $G_{\text{sum}(c)}(r)$

sum(c)
27

▶ Other example:

▶ $G_{\text{avg}(\text{salary})}(\text{instructor})$

▶ $G_{\text{count-distinct}(ID)}(\sigma_{\text{semester}=\text{"Spring"} \wedge \text{year}=2010}(\text{teaches}))$



Aggregation Operation - Example

- Find the average *salary* in each *department*.

$dept_name \mathrel{G} \text{avg}(salary) (instructor)$

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
22222	Einstein	Physics	95000

<i>dept_name</i>	<i>salary</i>
Biology	72000
Comp. Sci.	77333
Elec. Eng.	80000
Finance	85000
History	61000
Music	40000
Physics	91000



Aggregation Operation (Cont.)

- ▶ Result of aggregation *does not have a name*.
- ▶ We can use *rename operation* to give it a name.
- ▶ For convenience, we permit *renaming as part of aggregate operation*:

dept_name $\mathcal{G}$ *avg(salary)* *as avg_sal* (*instructor*)



Multiset Relational Algebra

- ▶ **Pure relational algebra** removes all duplicates (e.g. after projection).
- ▶ **Multiset relational algebra (多重集关系代数)** *retains duplicates to match SQL semantics* (SQL duplicate retention was initially for *efficiency*, but is now a *feature*).
- ▶ The basic operations in the multiset relational algebra defined as follows:
 - ▶ **Selection**: has as many duplicates of a tuple as in the input, if the tuple satisfies the selection
 - ▶ **Projection**: one tuple per input tuple, even if it is a duplicate
 - ▶ **Cartesian Product**: if there are m copies of t_1 in r , and n copies of t_2 in s , there are $m*n$ copies of $t_1.t_2$ in $r \times s$.



SQL and Relational Algebra

► **select** $A_1, A_2, \dots, A_n$
from $r_1, r_2, \dots, r_m$
where P

is equivalent to the following expression in multiset relational algebra:

$$\Pi_{A_1, \dots, A_n} (\sigma_P (r_1 \times r_2 \times \dots \times r_m))$$

► **select** $A_1, A_2, \text{sum}(A_3)$
from $r_1, r_2, \dots, r_m$
where P
group by A_1, A_2

is equivalent to the following expression in multiset relational algebra:

$$A_1, A_2 \mathcal{G}_{\text{sum}(A_3)} (\sigma_P (r_1 \times r_2 \times \dots \times r_m))$$



SQL and Relational Algebra

- ▶ More generally, the *non-aggregated attributes* in the **select clause** may be a subset of the **group by attributes**, in which case the equivalence is as follows:

select A_1 , **sum**(A_3)
from $r_1, r_2, \dots, r_m$
where P
group by A_1, A_2

is equivalent to the following expression in multiset relational algebra:

$$\Pi_{A_1, \text{sum}A_3}(\Pi_{A_1, A_2} \mathcal{G}_{\text{sum}(A_3) \text{ as sum}A_3}(\sigma_P(r_1 \times r_2 \times \dots \times r_m)))$$

Quiz

- ▶ Write the following queries in **relational algebra**, using the university schema.
- 1. Find the *titles* of *courses* in the Comp. Sci. *department* that have 3 *credits*.
- 2. Find the highest *salary* of any *instructor*.
- 3. Find the *IDs* of all *students* who have *taken* at least one Comp. Sci. *course*.

course(*course_id*, *title*, *dept_name*, *credits*)
instructor(*ID*, *name*, *dept_name*, *salary*)
section(*course_id*, *sec_id*, *semester*, *year*, *building*, *room_number*, *time_slot_id*)
teaches(*ID*, *course_id*, *sec_id*, *semester*, *year*)
student(*ID*, *name*, *dept_name*, *tot_cred*)
takes(*ID*, *course_id*, *sec_id*, *semester*, *year*, *grade*)



Answer for Quiz

1. $\Pi_{\text{title}} (\sigma_{\text{dept_name} = \text{'Comp. Sci'} \wedge \text{credits}=3} (\text{course}))$
2. $G_{\max(\text{salary})}(\text{instructor})$
3. $\Pi_{\text{ID}} (\sigma_{\text{dept_name}=\text{'Comp. Sci'}}(\text{takes} \bowtie \text{course}))$

course(course_id, title, dept_name, credits)

instructor(ID, name, dept_name, salary)

section(course_id, sec_id, semester, year, building, room_number, time_slot_id)

teaches(ID, course_id, sec_id, semester, year)

student(ID, name, dept_name, tot_cred)

takes(ID, course_id, sec_id, semester, year, grade)



The Tuple Relational Calculus

- ▶ The **tuple relational calculus** is a **nonprocedural query language**, which describes the *desired information without giving a specific procedure for obtaining that information*.
- ▶ A tuple-relational-calculus expression is of the form:
$$\{t \mid P(t)\}$$
- ▶ It is **the set of all tuples t** such that **predicate P is true for t** .
 - ▶ t is a tuple variable, $t[A]$ denotes the value of tuple t on attribute A .
 - ▶ $t \in r$ denotes that tuple t is in relation r
 - ▶ P is a *formula* similar to that of the predicate calculus.



Predicate Calculus Formula

- ▶ Set of attributes and constants
- ▶ Set of comparison operators: $<, \leq, =, \neq, >, \geq$
- ▶ Set of connectives: and ($\wedge$), or ($\vee$), not ($\neg$)
- ▶ **Implication** ($\Rightarrow$):

$x \Rightarrow y$, if x is true, then y is true (注: $x \Rightarrow y$ 只是说不会同时有 x 真与 y 假, 即 $\neg(x \wedge \neg y)$)

$$x \Rightarrow y \equiv \neg x \vee y$$

$\neg A \vee B$	A	B	$A \Rightarrow B$
T	T	T	T
F	T	F	F
T	F	T	T
T	F	F	T

- ▶ Set of quantifiers:

▶ $\exists t \in r (Q(t)) \equiv$ “*there exists a tuple t in relation r such that predicate $Q(t)$ is true*”

▶ $\forall t \in r (Q(t)) \equiv$ “ *Q is true for all tuples t in relation r* ”



Example Queries

- ▶ Find the *ID*, *name*, *dept_name*, *salary* for *instructors* whose *salary* is greater than \$80,000.

$$\{t \mid t \in \textit{instructor} \wedge t[\textit{salary}] > 80000\}$$

- ▶ Now output only the *ID* attribute value, rather than all attributes of the *instructor* relation.

$$\{t \mid \exists s \in \textit{instructor} (t[\textit{ID}] = s[\textit{ID}] \wedge s[\textit{salary}] > 80000)\}$$

- ▶ The set of **all tuples t** such that there **exists a tuple s** in relation *instructor* for which the values of t and s for the *ID* attribute are equal, and the value of s for the *salary* attribute is greater than \$80,000.
- ▶ Tuple variable t is defined on **only the *ID* attribute**, since that is the only attribute having a condition specified for t . Thus, the result is a relation on *ID*.



Example Queries

- ▶ Find the *names* of all *instructors* whose *department* is in the Watson *building*.

$$\{t \mid \exists s \in instructor (t [name] = s [name] \\ \wedge \exists u \in department (u [dept_name] = s [dept_name] \\ \wedge u [building] = \text{“Watson”}))\}$$

- ▶ It involves two relations: *instructor* and *department*.
- ▶ There are **two “there exists”** clauses in the expression, connected by *and* ($\wedge$).
- ▶ **Tuple variable u** is restricted to *departments* that are located in the Watson *building*, while **tuple variable s** is restricted to *instructors* whose *dept_name* matches that of tuple variable u .



Example Queries

- ▶ Find the set of all *courses* taught in the Fall 2009 semester, **or** the Spring 2010 semester, **or** both.

$$\{t \mid \exists s \in \text{section } (t[\text{course_id}] = s[\text{course_id}] \\ \wedge s[\text{semester}] = \text{"Fall"} \wedge s[\text{semester}] = 2009) \\ \vee \exists u \in \text{section } (t[\text{course_id}] = u[\text{course_id}] \\ \wedge u[\text{semester}] = \text{"Spring"} \wedge u[\text{semester}] = 2010)\}$$

- ▶ In the relational algebra, we used the **union** operation;
- ▶ In the tuple relational calculus, we use **two “there exists”** clauses, connected by *or* ($\vee$).
- ▶ If the same *course* is offered in both the Fall 2009 and Spring 2010 semesters, its *course_id* appears **only once in the result**, because the mathematical definition of a set does not allow duplicate members.



Example Queries

- ▶ Find the set of all *courses* taught in both the Fall 2009 semester **and** the Spring 2010 semester.
 - ▶ change the **or** ($\vee$) to **and** ($\wedge$) in the expression
$$\{t \mid \exists s \in \text{section } (t[\text{course_id}] = s[\text{course_id}] \wedge s[\text{semester}] = \text{"Fall"} \wedge s[\text{semester}] = 2009) \wedge \exists u \in \text{section } (t[\text{course_id}] = u[\text{course_id}] \wedge u[\text{semester}] = \text{"Spring"} \wedge u[\text{semester}] = 2010)\}$$
- ▶ Find the set of all *courses* taught in the Fall 2009 semester, **but not in** the Spring 2010 semester.
 - ▶ use the **not** ($\neg$) symbol
$$\{t \mid \exists s \in \text{section } (t[\text{course_id}] = s[\text{course_id}] \wedge s[\text{semester}] = \text{"Fall"} \wedge s[\text{semester}] = 2009) \wedge \neg \exists u \in \text{section } (t[\text{course_id}] = u[\text{course_id}] \wedge u[\text{semester}] = \text{"Spring"} \wedge u[\text{semester}] = 2010)\}$$



Example Queries

- ▶ Find all *students* who have taken all *courses* offered in the Biology *department*.
- ▶ **Implication ($\Rightarrow$)**
 - ▶ The formula $P \Rightarrow Q$ means “*P implies Q*”; that is, “*if P is true, then Q must be true.*”
 - ▶ $P \Rightarrow Q$ is logically equivalent to $\neg P \vee Q$.
- ▶ **“for all” construct ($\forall$)**
 - ▶ The notation $\forall t \in r (Q(t))$ means “*Q is true for all tuples t in relation r.*”



Example Queries

student(ID, name, dept_name, tot_cred)
course(course_id, title, dept_name, credits)
takes(ID, course_id, sec_id, semester, year, grade)

- Find all *students* who have taken all *courses* offered in the Biology department

$$\{t \mid \exists r \in \text{student} (r[ID] = t[ID]) \wedge$$
$$(\forall u \in \text{course} (u[\text{dept_name}] = \text{"Biology"})$$
$$\Rightarrow \exists s \in \text{takes} (t[ID] = s[ID] \wedge s[\text{course_id}] = u[\text{course_id}]))\}$$

- Interpretation:** The set of **all students** (that is, (*ID*) tuples *t*) such that, for **all tuples** *u* in the *course* relation, **if** the value of *u* on attribute *dept_name* is 'Biology', then **there exists** a tuple in the *takes* relation that includes the student *ID* and the *course_id*.
- If there is no *course* offered in the Biology department, **all student IDs satisfy the condition**.
- If without $\exists r \in \text{student} (r[ID] = t[ID])$, any value of *t* (including values that are not *student IDs* in the *student* relation) would qualify



Formal Definition of Tuple Relational Calculus

▶ $\{t \mid P(t)\}$

- ▶ Several *tuple variables* may appear in a *formula* P . A tuple variable is said to be a *free variable* unless it is quantified by a $\exists$ or $\forall$ (a *bound* variable).
- ▶ P is built up out of *atoms*
 - ▶ An atom has one of the following forms
 - ▶ $s \in r$, s is a *tuple variable* and r is a *relation* ($\notin$ is not allowed).
 - ▶ $s[x] \Theta u[y]$, s and u are *tuple variables*, x is an *attribute* on which s is defined, y is an *attribute* on which u is defined, Θ is a *comparison operator* ($<, \leq, =, \neq, >, \geq$).
 - ▶ $s[x] \Theta c$, s is a *tuple variable*, x is an *attribute* on which s is defined, Θ is a *comparison operator*, and c is a *constant* in the *domain of attribute* x .



Formal Definition of Tuple relational calculus

- ▶ **Rules** for building up formulae from atoms
 - ▶ An **atom** is a formula.
 - ▶ If P_1 is a formula, then so are $\neg P_1$ and (P_1) .
 - ▶ If P_1 and P_2 are formulae, then so are $P_1 \vee P_2$, $P_1 \wedge P_2$, and $P_1 \Rightarrow P_2$.
 - ▶ If $P_1(s)$ is a formula containing a free tuple variable s , and r is a relation, then $\exists s \in r (P_1(s))$ and $\forall s \in r (P_1(s))$ are also formulae.
- ▶ **Equivalent expressions** in the tuple relational calculus
 - ▶ $P_1 \wedge P_2$ is equivalent to $\neg (\neg(P_1) \vee \neg(P_2))$.
 - ▶ $\forall t \in r (P_1(t))$ is equivalent to $\neg \exists t \in r (\neg P_1(t))$.
 - ▶ $P_1 \Rightarrow P_2$ is equivalent to $\neg(P_1) \vee P_2$.



Safety of Expressions

- ▶ It is possible to write tuple calculus expressions that **generate infinite relations**.
 - ▶ For example, $\{ t \mid \neg t \in r \}$ results in an **infinite relation** if the *domain of any attribute of relation r is infinite*.
- ▶ To guard against the problem, we **restrict the set of allowable expressions to safe expressions**.
- ▶ **Domain** of a tuple relational formula, P , is *the set of all values referenced by P* , denoted **$dom(P)$** .
 - ▶ They include *values mentioned in P* itself, as well as *values that appear in a tuple of a relation mentioned in P* .
 - ▶ E.g. $dom(t \in instructor \wedge t[salary] > 80000)$ is the set containing 80000 as well as the set of all values appearing in any attribute of any tuple in the *instructor* relation.



Safety of Expressions

- ▶ An expression $\{t \mid P(t)\}$ in the tuple relational calculus is **safe** if *all values that appear in the result are values from $\text{dom}(P)$* .
- ▶ The expression $\{t \mid \neg(t \in \text{instructor})\}$ is not safe.
 - ▶ $\text{dom}(\neg(t \in \text{instructor}))$ is the set of all values appearing in *instructor*.
 - ▶ However, it is possible to have a tuple t not in *instructor* that contains values that do not appear in *instructor*.



Safety of Expressions

- Find all *students* who have *taken* all *courses* offered in the *Biology department*.

$$\{t \mid \exists r \in \text{student} (r[ID] = t[ID]) \\ \wedge (\forall u \in \text{course} (u[\text{dept_name}] = \text{“Biology”}) \Rightarrow \\ \exists s \in \text{takes} (t[ID] = s[ID] \wedge s[\text{course_id}] = u[\text{course_id}])))\}$$

- Note that without the *existential quantification on student*, the above query would be **unsafe** if the *Biology department* has not offered any courses.
- If there is no *course* offered in the *Biology department*, all *student IDs* (including values that are not *student IDs* in the *student* relation) satisfy the condition.



Domain Relational Calculus

- ▶ **Domain Relational Calculus** is a *nonprocedural query* language equivalent in power to the tuple relational calculus.
- ▶ Each query is an expression of the form:

$$\{ \langle x_1, x_2, \dots, x_n \rangle \mid P(x_1, x_2, \dots, x_n) \}$$

- ▶ $x_1, x_2, \dots, x_n$ represent *domain variables*.
- ▶ P represents *a formula* similar to that of the tuple relational calculus.
- ▶ As a notational shorthand, we write $\exists a, b, c (P(a, b, c))$ for $\exists a (\exists b (\exists c (P(a, b, c))))$.



Example Queries

- Find the *instructor ID*, *name*, *dept_name*, and *salary* for *instructors* whose *salary* is greater than \$80,000.

$\{ \langle i, n, d, s \rangle \mid \langle i, n, d, s \rangle \in \text{instructor} \wedge s > 80000 \}$

instructor(ID, name, dept_name, salary)

- Find all *instructor name* for *instructors* whose *salary* is greater than \$80,000.

$\{ \langle n \rangle \mid \exists i, d, s (\langle i, n, d, s \rangle \in \text{instructor} \wedge s > 80000) \}$

instructor(ID, name, dept_name, salary)

- Find the *names* of all *instructors* in the Physics *department* together with the *course_id* of all *courses* they *teach*.

$\{ \langle n, c \rangle \mid \exists i, a (\langle i, c, a, s, y \rangle \in \text{teaches}$

$\wedge \exists d, s (\langle i, n, d, s \rangle \in \text{instructor} \wedge d = \text{“Physics”})) \}$

teaches(ID, course_id, sec_id, semester, year)



Example Queries

- ▶ Find the set of all *courses taught* in the Fall 2009 semester, the Spring 2010 semester, or both.

section(course_id, sec_id, semester, year, building, room_number, time_slot_id)

$$\{ \langle c \rangle \mid \exists a, s, y, b, r, t (\langle c, a, s, y, b, r, t \rangle \in \text{section} \\ \wedge s = \text{"Fall"} \wedge y = \text{"2009"}) \\ \vee \exists a, s, y, b, r, t (\langle c, a, s, y, b, r, t \rangle \in \text{section} \\ \wedge s = \text{"Spring"} \wedge y = \text{"2010"}) \}$$



Safety of Expressions

- ▶ The expression

$$\{ \langle x_1, x_2, \dots, x_n \rangle \mid P(x_1, x_2, \dots, x_n) \}$$

is **safe** if *all* of the following hold:

- ▶ *All values* that appear in tuples of the expression are values from *dom* (*P*) (that is, the values appear either in *P* or in a tuple of a relation mentioned in *P*).
- ▶ For *every “there exists”* subformula of the form $\exists x (P_1(x))$, the subformula is true if and only if there is *a value of x* in *dom* (*P*₁) such that *P*₁(*x*) is true.
- ▶ For *every “for all”* subformula of the form $\forall x (P_1(x))$, the subformula is true if and only if *P*₁(*x*) is true for *all values x* from *dom* (*P*₁).



Safety of Expressions

- Find all *students* who have *taken* all *courses* offered in the *Biology department*.

course(*course_id*, *title*, *dept_name*, *credits*)

takes(*ID*, *course_id*, *sec_id*, *semester*, *year*, *grade*)

$$\begin{aligned} & \{ \langle i \rangle \mid \exists n, d, tc (\langle i, n, d, tc \rangle \in \text{student}) \wedge \\ & \quad \forall ci, ti, dn, cr (\langle ci, ti, dn, cr \rangle \in \text{course} \wedge dn = \text{“Biology”} \\ & \Rightarrow \\ & \quad \exists si, se, y, g (\langle i, ci, si, se, y, g \rangle \in \text{takes})) \} \end{aligned}$$

- Note that without the *existential quantification on student*, the above query would be **unsafe** if the *Biology* department has not offered any courses.



Homework

► 6.14, 6.15

6.14 Consider the following relational schema for a library:

member(*memb_no*, *name*, *dob*)
books(*isbn*, *title*, *authors*, *publisher*)
borrowed(*memb_no*, *isbn*, *date*)

Write the following queries in relational algebra.

- Find the names of members who have borrowed any book published by “McGraw-Hill”.
- Find the name of members who have borrowed all books published by “McGraw-Hill”.
- Find the name and membership number of members who have borrowed more than five different books published by “McGraw-Hill”.
- For each publisher, find the name and membership number of members who have borrowed more than five books of that publisher.
- Find the average number of books borrowed per member. Take into account that if an member does not borrow any books, then that member does not appear in the *borrowed* relation at all.



Homework

- 6.15 Consider the employee database of Figure 6.22. Give expressions in tuple relational calculus and domain relational calculus for each of the following queries:
- Find the names of all employees who work for “First Bank Corporation”.
 - Find the names and cities of residence of all employees who work for “First Bank Corporation”.
 - Find the names, street addresses, and cities of residence of all employees who work for “First Bank Corporation” and earn more than \$10,000.
 - Find all employees who live in the same city as that in which the company for which they work is located.
 - Find all employees who live in the same city and on the same street as their managers.
 - Find all employees in the database who do not work for “First Bank Corporation”.
 - Find all employees who earn more than every employee of “Small Bank Corporation”.
 - Assume that the companies may be located in several cities. Find all companies located in every city in which “Small Bank Corporation” is located.

employee (person_name, street, city)

works (person_name, company_name, salary)

company (company_name, city)

manages (person_name, manager_name)

